

```

# vim: set syntax=nginx :
user "www";
worker_processes 1;
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

# Main listening ports
listen 80;
listen 8080;

# Default server configuration
server {
    listen 80;
    server_name _;
    root /var/www/html;
    index index.html index.htm;

    # Redirect all requests
    return 301 https://$host$request_uri;
}

server {
    listen 8080;
    server_name _;
    root /var/www/html;
    index index.html index.htm;

    # Redirect all requests
    return 301 https://$host$request_uri;
}

```

Figure 1: Pound configuration—I

```

# vim: set syntax=nginx :
user "www";
worker_processes 1;
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

# Main listening ports
listen 80;
listen 8080;

# Default server configuration
server {
    listen 80;
    server_name _;
    root /var/www/html;
    index index.html index.htm;

    # Redirect all requests
    return 301 https://$host$request_uri;
}

server {
    listen 8080;
    server_name _;
    root /var/www/html;
    index index.html index.htm;

    # Redirect all requests
    return 301 https://$host$request_uri;
}

```

Figure 2: Pound configuration—II

```

<VirtualHost 192.168.0.7:80>
    DocumentRoot /var/www/html/site1
    ServerName www.site1.com
</VirtualHost>
<VirtualHost 192.168.0.8:80>
    DocumentRoot /var/www/html/site2
    ServerName www.site2.com
</VirtualHost>

```

- Start your Web server with `service httpd start`; access both websites and make sure they are working.

Here, notice that in `httpd.conf` I used two different names for `ServerName`—one is `www.site1.com` and the other `www.site2.com`. Also remember that the contents of our two `index.html` files differ. I'll explain the reason later in this article.

Now get back to the first machine, where we'll configure Pound—which is the easiest part of the whole process. Edit `/etc/pound/pound.conf` and on line 6 you'll see the main listening ports setting `ListenHTTP`. Here, give the port number and IP address of the machine on which you're running Pound. You can see my settings in Figure 1.

Now, you'll need to provide the addresses of the Web servers you're running. On `pound.conf` line 35, you'll find `Catch-all`

`server(s)`; in two `BackEnd` blocks below, give the IP addresses of the Web servers, port numbers and priorities. For an example, see Figure 2. I have set the `priority` to 7 for the 192.168.0.7 Web server, and for 192.168.0.8, it's 3. This means that when a request is received to access the website, there are 70 per cent chances that the request will be sent to the first server. You can omit the `Priority` field to equally distribute the traffic to both servers.

Now start the service with `service pound start` and if everything is done right, the service will start smoothly.

Well, now it's time to check by accessing the site. Prior to that, you may need to adjust your firewall settings; either make the necessary changes, or temporarily stop the firewall service using `service iptables stop` on both the machines.

Open your browser and type the IP address of the machine on which you're running Pound (for me, it's 192.168.0.6) and you'll see the content "SITE1"; now just click the *Refresh* button a few times and you'll see the contents changing.

Observations and explanations

First of all, I'm running my Web servers on 192.168.0.7 and 192.168.0.8 as visible above—yet, I access them at the address 192.168.0.6. This is because your (client/browser) request(s) must be made to Pound, which fetches the response from the Web servers mentioned in `pound.conf` and returns it. Sure, in our test set-up, you can directly access the websites at 192.168.0.7 or .8, but in real-world situations, you'd firewall these back-end servers to only accept requests from the Pound server, and ensure that all client requests must be made to the server running Pound.

Second, why do the responses differ (*SITE1* first, and when you *Refresh* a few times, *SITE2*)? Pound distributes incoming requests to either of the configured back-end servers, based on their priority. As noted earlier, we deliberately entered different content in each `index.html` file so we'd know which site the response came from. However, in an actual working environment, all Web servers should be serving the same content (if it's static content, that is).

Remember our `www.site1.com` and `www.site2.com` virtual host site names? They are obsolete or unused; all access being through Pound, the only address that matters is that of the machine running the Pound service.

That's it!

This is just a beginning; you can further extend this exercise by implementing HTTPS or deploying a Squid proxy server for your load-balanced Web servers. I hope you're feeling a bit more comfortable with the term 'load balancing...' 

By: Vinayak Pandey

A computer graduate, a Red Hat Certified Engineer on RHEL6 and an open source enthusiast, the author has also worked as a freelance Linux trainer for a while. On the professional front, he will start his journey with Tata Consultancy Services soon. You can catch him at <https://hacktips.wordpress.com>.